



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ДОМАШНЯЯ РАБОТА

«Реализация алгоритма поиска с опечатками»

ДИСЦИПЛИНА: «Архитектура автоматизированных систем
обработки информации и управления»

Выполнил: студент гр. ИУ5-24Б

_____ (Бондаренко Д.А.)
(Подпись) (Ф.И.О.)

Проверил:

_____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

Введение

Актуальность темы

В современных информационных системах, обрабатывающих большие объемы текстовой информации, часто возникают ситуации, когда входные данные содержат опечатки. Это может быть связано с человеческим фактором (при ручном вводе), ошибками распознавания или несовершенством каналов передачи данных. Для обеспечения устойчивости систем к таким ошибкам применяются алгоритмы нечеткого поиска.

Цель работы:

Целью данной работы является разработка программного модуля для консольного приложения на языке C#, реализующего алгоритм вычисления редакционного расстояния Дамерау-Левенштейна и демонстрирующего его работу в интерактивном режиме.

Задачи:

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ существующих метрик сравнения строк, обосновать выбор метрики Дамерау-Левенштейна.
2. Изучить математический аппарат алгоритма Вагнера-Фишера и адаптировать его для учета операции транспозиции символов.
3. Спроектировать структуру консольного приложения с бесконечным циклом обработки пользовательского ввода.
4. Реализовать функции вычисления расстояния и вывода результатов в строгом соответствии с синтаксисом языка C#.
5. Провести верификацию разработанного алгоритма путем сравнения полученных результатов с эталонными примерами из методической литературы.

Теоретическая часть

Редакционное расстояние как метрика

Под редакционным расстоянием понимается минимальное количество элементарных операций, необходимых для преобразования одной строки символов в другую. Чем меньше это расстояние, тем более похожими считаются строки. В классической постановке (расстояние Левенштейна) допускаются три типа операций: вставка (Insert), удаление (Delete) и замена (Replace), каждая из которых имеет стоимость, равную единице.

Расширение Дамерау

Фредерик Дамерау, исследуя статистику реальных опечаток, установил, что наиболее частой ошибкой при наборе текста является перестановка двух

соседних символов (транспозиция). В классическом алгоритме такая ошибка требует двух операций замены, что не соответствует интуитивному восприятию сложности. Поправка Дамерау вводит четвертую операцию — транспозицию (Transposition), стоимость которой также принимается равной единице. Это позволяет более точно оценивать схожесть строк для приложений, связанных с поиском и исправлением опечаток.

Обоснование выбора метрики

Для решения поставленной задачи (поиск с опечатками) использование расстояния Дамерау-Левенштейна является более предпочтительным по сравнению с классическим расстоянием Левенштейна, так как оно лучше моделирует реальные ошибки пользователя, особенно при работе с клавиатурным вводом.

Проектирование и реализация

Архитектура приложения

Приложение разработано как консольное с использованием платформы .NET. Для обеспечения простоты и наглядности был выбран подход с единым файлом исходного кода, содержащим класс `Program` с точкой входа `Main`.

Алгоритм Вагнера-Фишера с поправкой на транспозицию

В основе реализации лежит метод динамического программирования. Для строк длиной M и N создается матрица размером $(M+1) \times (N+1)$. Значение $D[i, j]$ интерпретируется как минимальное расстояние между префиксами $S1[0..i-1]$ и $S2[0..j-1]$. Инициализация матрицы производится следующим образом:

- $D[0, 0] = 0$ (обе строки пусты)
- $D[i, 0] = i$ (удаление всех символов из первой строки)
- $D[0, j] = j$ (вставка всех символов во вторую строку)

Заполнение остальных ячеек осуществляется по рекуррентному соотношению, учитывающему операции вставки, удаления, замены и, при соблюдении дополнительных условий, транспозиции.

Особенности программной реализации

Код программы включает в себя несколько ключевых аспектов:

- Регистронезависимость: Перед вычислением обе строки приводятся к верхнему регистру.
- Обработка граничных случаев: Проверка входных параметров на `null` и обработка пустых строк.
- Точное следование спецификации: Использование метода `Substring` для посимвольного сравнения, как это указано в оригинальном листинге методических указаний.

- Интерактивный цикл: Основной цикл программы запрашивает у пользователя две строки, выводит результат и завершается только при вводе команды "exit".

Листинг программы

```
using System;

class Program
{
    static int Distance(string str1Param, string str2Param)
    {
        if ((str1Param == null) || (str2Param == null))
            return -1;

        int str1Len = str1Param.Length;
        int str2Len = str2Param.Length;

        if ((str1Len == 0) && (str2Len == 0))
            return 0;
        if (str1Len == 0)
            return str2Len;
        if (str2Len == 0)
            return str1Len;

        string str1 = str1Param.ToUpper();
        string str2 = str2Param.ToUpper();

        int[,] matrix = new int[str1Len + 1, str2Len + 1];

        for (int i = 0; i <= str1Len; i++)
            matrix[i, 0] = i;
        for (int j = 0; j <= str2Len; j++)
            matrix[0, j] = j;

        for (int i = 1; i <= str1Len; i++)
        {
            for (int j = 1; j <= str2Len; j++)
            {
                int symbEqual = (str1.Substring(i - 1, 1) == str2.Substring(j - 1, 1)) ? 0 :
1;

                int ins = matrix[i, j - 1] + 1;
                int del = matrix[i - 1, j] + 1;
                int subst = matrix[i - 1, j - 1] + symbEqual;

                matrix[i, j] = Math.Min(Math.Min(ins, del), subst);
            }
        }
    }
}
```

```

        if ((i > 1) && (j > 1) &&
            (str1.Substring(i - 1, 1) == str2.Substring(j - 2, 1)) &&
            (str1.Substring(i - 2, 1) == str2.Substring(j - 1, 1)))
        {
            matrix[i, j] = Math.Min(matrix[i, j],
                matrix[i - 2, j - 2] + symbEqual);
        }
    }
}
return matrix[str1Len, str2Len];
}

static void WriteDistance(string str1Param, string str2Param)
{
    int d = Distance(str1Param, str2Param);
    Console.WriteLine(""" + str1Param + "", "" + str2Param + "" -> " +
d.ToString());
}

static void Main()
{
    while (true)
    {
        Console.WriteLine("\nПоиск с опечатками (расстояние Дамерау-
Левенштейна) ");
        Console.Write("Введите первую строку (или 'exit' для выхода): ");
        string? str1 = Console.ReadLine();

        if (str1?.ToLower() == "exit")
        {
            Console.WriteLine("Программа завершена.");
            break;
        }

        Console.Write("Введите вторую строку: ");
        string? str2 = Console.ReadLine();

        if (str1 == null || str2 == null)
        {
            Console.WriteLine("Ошибка ввода.");
            continue;
        }

        WriteDistance(str1, str2);
    }
}

```

```
}  
}  
}
```

Тестирование и результаты

Методика тестирования

Для проверки корректности работы алгоритма использовался метод функционального тестирования. Были подготовлены тестовые наборы, каждый из которых нацелен на проверку определенного аспекта алгоритма:

- Проверка базовых операций (вставка, удаление, замена).
- Проверка операции транспозиции.
- Проверка комбинаций различных операций.
- Проверка граничных условий (пустые строки, одинаковые строки).

Результаты выполнения

В ходе тестирования были получены следующие результаты:

Тестовый случай (S1, S2)	Ожидаемый результат	Полученный результат
("", "")	0	0
("A", "")	1	1
("", "B")	1	1
("пример", "1пример")	1	1
("приМер", "прМир")	2	2

Результаты работы программы представлен на рисунке 1

```
Поиск с опечатками (расстояние Дамерау-Левенштейна)  
Введите первую строку (или 'exit' для выхода): приМер  
Введите вторую строку: прМир  
'приМер', 'прМир' -> 2  
  
Поиск с опечатками (расстояние Дамерау-Левенштейна)  
Введите первую строку (или 'exit' для выхода):
```

Рисунок 1 -Результаты работы программы

Заключение

Итоги работы

В рамках выполнения домашнего задания была разработана и протестирована программа, реализующая алгоритм поиска с опечатками. Программа полностью соответствует функциональным требованиям:

- Обеспечивает бесконечный цикл ввода до команды "exit".
- Корректно вычисляет расстояние Дамерау-Левенштейна.
- Выводит результаты в понятном для пользователя формате.

Выводы по теоретической части

В ходе работы были углублены знания в области обработки текстовой информации. Изучены математические основы редакционного расстояния, выявлены преимущества метрики Дамерау-Левенштейна перед классической метрикой Левенштейна для задач нечеткого поиска. Освоен метод динамического программирования на примере алгоритма Вагнера-Фишера.

Возможные пути улучшения

Несмотря на то, что разработанная программа полностью удовлетворяет поставленным требованиям, можно выделить направления для ее потенциального улучшения:

- Оптимизация памяти: Для снижения потребления памяти можно хранить не всю матрицу, а только две последние строки, однако это потребует модификации логики обработки транспозиции.
- Расширение функционала: Добавление возможности поиска слова в заданном словаре с выводом всех вариантов, расстояние до которых не превышает заданный порог.
- Улучшение пользовательского интерфейса: Реализация более дружелюбного интерфейса с цветовым выделением различий между строками.